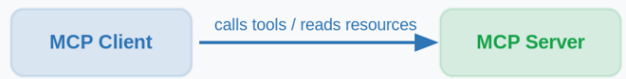
Client Primitives Agenda

- Sampling
- Elicitation
- Roots
- Demo

Overview — The Direction Flips

Earlier modules we have covered one direction: the Client calls the Server — to list tools, read resources, and get prompts.



This Module introduces the reverse direction: capabilities the Client exposes so the Server can call back into it.

There are three such client-side capabilities:

- **Sampling** — server asks the client to run an LLM completion
- **Elicitation** — server asks the user for structured input
- **Roots** — client tells the server which folders to focus on

1. Sampling

Sampling lets a server request a language model completion through the client. The server does not need its own API key or model subscription — the client, which already has model access, does the inference on the server's behalf.

Why is this useful?

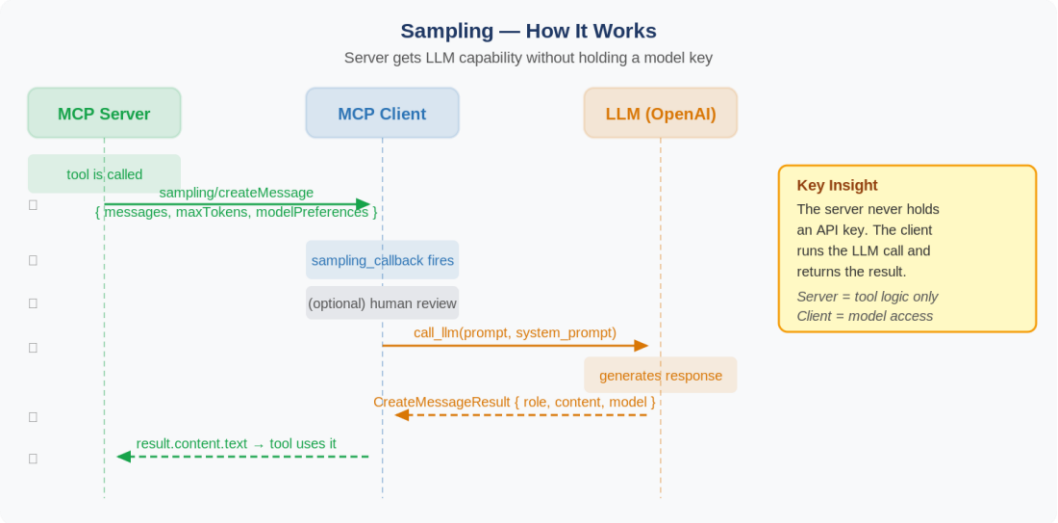
Imagine you are building an MCP tool that needs to generate a product description. Without sampling:

- The server would need its own OpenAI/Anthropic key
- Every deployment would need its own billing and key management
- The server author must decide which model to use

With sampling:

- The server just asks: "please generate this for me"
- The client runs the LLM using its own key and model
- The user stays in control — they can review or deny the request

The Flow



The Sampling Request

When the server calls **ctx.session.create_message(),** it sends a request like this:

```
{
  "messages": [
    {
      "role": "user",
      "content": "Create a product description about EcoBottle described as sustainable, stainless steel, keeps drinks cold 24h"
    }
  ],
  "modelPreferences": {
    "hints": [{ "name": "gpt-5.6-luna" }],
    "costPriority":       0.3,
    "speedPriority":    0.2,
    "intelligencePriority": 0.9
  },
  "systemPrompt": "You are a helpful assistant. Create a compelling product description.",
  "maxTokens":    1500
}
```

modelPreferences — the three priority weights

Priority	What it means	Example value
costPriority	How much to minimise API cost	0.3 (not a major concern)
speedPriority	How much to prioritise fast responses	0.2 (can wait for thoroughness)
intelligencePriority	How much to prioritise reasoning quality	0.9 (complex analysis needed)

The hints field suggests a preferred model by name, but the client is free to choose the actual model.

product_server.py— the tool that initiates sampling via **ctx.session.create_message()**

```
"""
product_server.py — the MCP server (FastMCP, stdio transport).

Three tools, each demonstrating a client primitive:

  create_product   -> SAMPLING     : server asks the client to run an LLM
  book_vacation    -> ELICITATION  : server asks the user for structured input
  recommend_flight -> BOTH         : elicit preferences, then sample the LLM

IMPORTANT: over stdio, this process's STDOUT is the MCP protocol channel.
Never print() to stdout here — all human-readable narration goes to STDERR
via log(), which shows up in the client's terminal without corrupting traffic.
"""

import json
import sys
from typing import Literal

from pydantic import BaseModel, Field

from mcp.server.mcpserver import MCPServer, Context
from mcp.server.session import ServerSession
from mcp.types import (
    SamplingMessage,
    TextContent,
)

mcp = MCPServer("product server")

def log(msg: str) -> None:
    """Narrate to STDERR (stdout is reserved for the MCP protocol)."""
    print(f"[SERVER] {msg}", file=sys.stderr, flush=True)

@mcp.tool()
async def create_product(
    product_name: str,
    keywords: str,
```

```
    ctx: Context[ServerSession, None],
) -> str:
    """Create a product and generate its description using LLM sampling."""
    prompt = (
        f"Create a product description about {product_name} "
        f"described as {keywords}"
    )

    log(f"create_product: requesting sampling for '{product_name}'...")
    # Server -> Client: ask the client's LLM to generate text. The server
    # has no model key — create_message() hands the work to the client.
    result = await ctx.session.create_message(
        messages=[
            SamplingMessage(
                role="user",
                content=TextContent(type="text", text=prompt),
            )
        ],
        system_prompt="You are a helpful assistant. Create a compelling product description.",
        max_tokens=200,
    )

    description = result.content.text
    log("create_product: received description, returning JSON.")
    return json.dumps({"name": product_name, "description": description}, indent=2)


if __name__ == "__main__":
    log("product-server starting on stdio...")
    mcp.run()  # stdio transport by default
```

product_client.py

```
import asyncio
import os
import sys
from openai import AsyncOpenAI
from dotenv import load_dotenv

load_dotenv()  # load .env file if present
```

```
from mcp import ClientSession, types
from mcp.client.stdio import stdio_client, StdioServerParameters
from mcp.client.session import ClientRequestContext

_API_KEY = os.getenv("OPENAI_API_KEY")

async def handle_sampling_message(
    context: ClientRequestContext,
    params: types.CreateMessageRequestParams,
) -> types.CreateMessageResult:
    print(
        f"[CLIENT] sampling_callback fired — the server asked for an LLM completion.",
        flush=True,
    )

    # Everything the server sent is in params.
    user_text = params.messages[0].content.text
    system_prompt = params.system_prompt or ""

    print(f"  calling LLM...", flush=True)
    client = AsyncOpenAI(api_key=_API_KEY)
    response = await client.chat.completions.create(
        model="gpt-5.6-luna",
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_text},
        ],
    )
    print(f"  returning the completion to the server.", flush=True)
    response = response.choices[0].message.content
    return types.CreateMessageResult(
        role="assistant",
        content=types.TextContent(type="text", text=response),
        model="gpt-5.6-luna",
        stopReason="endTurn",
    )

# ---------------------------------------------------------------------
# Tool invocations (each prints the live round-trip)
```

```
# ---------------------------------------------------------------------
def _result_text(result) -> str:
    parts = [b.text for b in result.content if isinstance(b, types.TextContent)]
    return "\n".join(parts)

async def demo_sampling(session: ClientSession) -> None:
    print("SAMPLING demo  ->  create_product")
    name = input("Product name [EcoBottle]: ").strip() or "EcoBottle"
    keywords = (
        input(
            "Keywords [sustainable, stainless steel, keeps drinks cold 24h]: "
        ).strip()
        or "sustainable, stainless steel, keeps drinks cold 24h"
    )
    print(f"[CLIENT] Calling tool create_product(name='{name}')...", flush=True)
    result = await session.call_tool(
        "create_product", {"product_name": name, "keywords": keywords}
    )
    print("\n--- tool result ---")
    print(_result_text(result))

async def main() -> None:
    server_params = StdioServerParameters(command=sys.executable, args=["product_server.py"])

    async with stdio_client(server_params) as (read, write):
        async with ClientSession(
            read,
            write,
            sampling_callback=handle_sampling_message,
            # elicitation_callback=handle_elicitation,
        ) as session:
            await session.initialize()
            print(f"[CLIENT] connected to product-server.", flush=True)

            while True:
                print(
                    "\nChoose a demo:\n"
                    "  1) Sampling     -> create_product\n"
                    "  2) Elicitation  -> launch_product\n"
```

```
            "  q) quit"
        )
        pick = input("> ").strip().lower()
        if pick == "1":
            await demo_sampling(session)
        elif pick == "2":
            print("Not Implemented yet")
        elif pick in ("q", "quit", "exit"):
            print("bye.", flush=True)
            break
        else:
            print("Pick 1, 2, or q.")

if __name__ == "__main__":
    asyncio.run(main())
```

Human-in-the-Loop

User Control

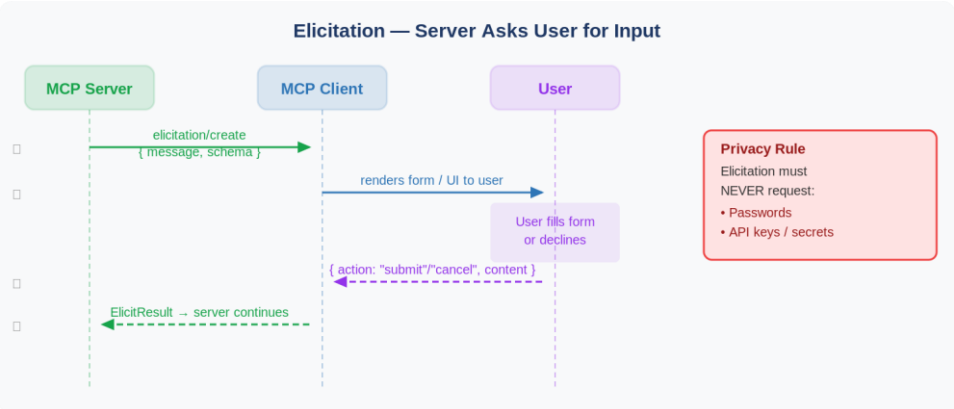
Sampling requests may require explicit user consent. The client can show the exact prompt, model, and token limit before forwarding to the LLM. Users can approve, deny, or modify.

Clients should also rate-limit sampling requests and validate message content.

2. Elicitation

Elicitation lets a server ask the user for specific information mid-interaction, instead of requiring everything upfront or failing when data is missing. The server sends a JSON schema; the client renders a form; the user fills it in or declines.

The Flow



Example — Barcelona Booking Confirmation

The server sends an elicitation request like this:

```
{
  "method": "elicitation/create",
  "params": {
    "message": "Please confirm launch details for 'EcoBottle':",
    "requested_schema": {
      "type": "object",
      "properties": {
        "confirmLaunch": {
          "type": "boolean",
          "description": "Confirm you want to launch this product"
        },
        "price": {
          "type": "number",
          "default": 29.99,
          "minimum": 0,
          "description": "Retail price (USD)"
        },
        "initialStock": {
          "type": "integer",
          "default": 100,
          "minimum": 0,
          "description": "Initial stock quantity"
        },
        "shippingSpeed": {
          "type": "string",
          "default": "standard",
          "enum": ["standard", "priority", "overnight"],
          "description": "Shipping speed to offer at launch"
        }
      },
      "required": ["confirmLaunch"]
    }
  }
}
```

The Response

The client returns one of three actions:

action value	Meaning
submit	User filled the form and confirmed
cancel	User explicitly cancelled
decline	User chose not to provide the information

Add the following to product_server.py

```
# ---------------------------------------------------------------------------
# Schema — FastMCP turns this Pydantic model into the JSON schema the client
# receives in the elicitation/create request.
# ---------------------------------------------------------------------------
class LaunchConfirmation(BaseModel):
    confirmLaunch: bool = Field(
        description="Confirm you want to launch this product"
    )
    price: float = Field(
        default=29.99, ge=0, description="Retail price (USD)"
    )
    initialStock: int = Field(
        default=100, ge=0, description="Initial stock quantity"
    )
    shippingSpeed: str = Field(
        default="standard",
        description="Shipping speed to offer at launch",
        json_schema_extra={"enum": ["standard", "priority", "overnight"]},
    )


# ---------------------------------------------------------------------------
# 2. ELICITATION — launch_product
# ---------------------------------------------------------------------------
@mcp.tool()
async def launch_product(
    product_name: str,
    ctx: Context[ServerSession, None],
) -> str:
    """Launch a product, confirming pricing and inventory with the user first."""
    log(f"launch_product: eliciting launch details for '{product_name}'...")
    # Server -> Client: elicitation/create goes out here. The client renders
```

```
    # a form from LaunchConfirmation and the user fills / declines / cancels.
    result = await ctx.elicit(
        message=f"Please confirm launch details for '{product_name}':",
        schema=LaunchConfirmation,
    )

    if result.action == "accept" and result.data:
        data = result.data
        if not data.confirmLaunch:
            return "Launch not confirmed by user."
        log("launch_product: user accepted, finalizing launch.")
        return (
            f"Launched {product_name}! "
            f"Price: ${data.price:.2f}, "
            f"Initial stock: {data.initialStock}, "
            f"Shipping: {data.shippingSpeed}."
        )
    elif result.action == "decline":
        log("launch_product: user declined.")
        return "User declined to provide launch details."
    else:  # "cancel"
        log("launch_product: user cancelled.")
        return "User cancelled the launch flow."
```

Add the following to product_client.py

```
# ---------------------------------------------------------------------------
# ELICITATION callback — the client renders a form and collects user input
# ---------------------------------------------------------------------------
async def handle_elicitation(
    context: ClientRequestContext,
    params: types.ElicitRequestParams,
) -> types.ElicitResult:
    print(
        f"[CLIENT] elicitation_callback fired — the server is asking the user for input.",
        flush=True,
    )

    print(f"\n  {params.message}")
    choice = input(
```

```
        "  Proceed? [Enter = fill form,  d = decline,  c = cancel]: "
    ).strip().lower()

    if choice == "d":
        print(f"[CLIENT]  user declined.", flush=True)
        return types.ElicitResult(action="decline")
    if choice == "c":
        print(f"[CLIENT]  user cancelled.", flush=True)
        return types.ElicitResult(action="cancel")

    answers = _fill_form()
    print(f"[CLIENT]  submitting the user's answers to the server.", flush=True)
    return types.ElicitResult(action="accept", content=answers)

def _fill_form() -> dict:
    """Prompt for the fields of LaunchConfirmation (console 'form')."""
    confirm = input("  - Confirm you want to launch this product (y/n): ").strip().lower()
    confirm_launch = confirm in ("y", "yes", "true", "1")

    price_raw = input("  - Retail price (USD) [default: 29.99]: ").strip()
    price = float(price_raw) if price_raw else 29.99

    stock_raw = input("  - Initial stock quantity [default: 100]: ").strip()
    initial_stock = int(stock_raw) if stock_raw else 100

    shipping_options = ["standard", "priority", "overnight"]
    print("  - Shipping speed to offer at launch [default: standard]")
    for i, opt in enumerate(shipping_options, 1):
        print(f"     {i}) {opt}")
    shipping_raw = input("    choose number (or Enter for default): ").strip()
    if shipping_raw.isdigit() and 1 <= int(shipping_raw) <= len(shipping_options):
        shipping_speed = shipping_options[int(shipping_raw) - 1]
    else:
        shipping_speed = "standard"

    return {
        "confirmLaunch": confirm_launch,
        "price": price,
        "initialStock": initial_stock,
```

```
    "shippingSpeed": shipping_speed,
  }



async def demo_elicitation(session: ClientSession) -> None:
  print("ELICITATION demo  ->  launch_product")
  name = input("Product name [EcoBottle]: ").strip() or "EcoBottle"
  print(f"[CLIENT] Calling tool launch_product(name='{name}')...", flush=True)
  result = await session.call_tool("launch_product", {"product_name": name})
  print("\n--- tool result ---")
  print(_result_text(result))
```

Edit Main Method:

```
if pick == "1":
    await demo_sampling(session)
elif pick == "2":
    await demo_elicitation(session)
elif pick in ("q", "quit", "exit"):
    print("bye.", flush=True)
    break
```

Privacy Rule — Critical

Elicitation must NEVER request passwords, API keys, or other secrets.

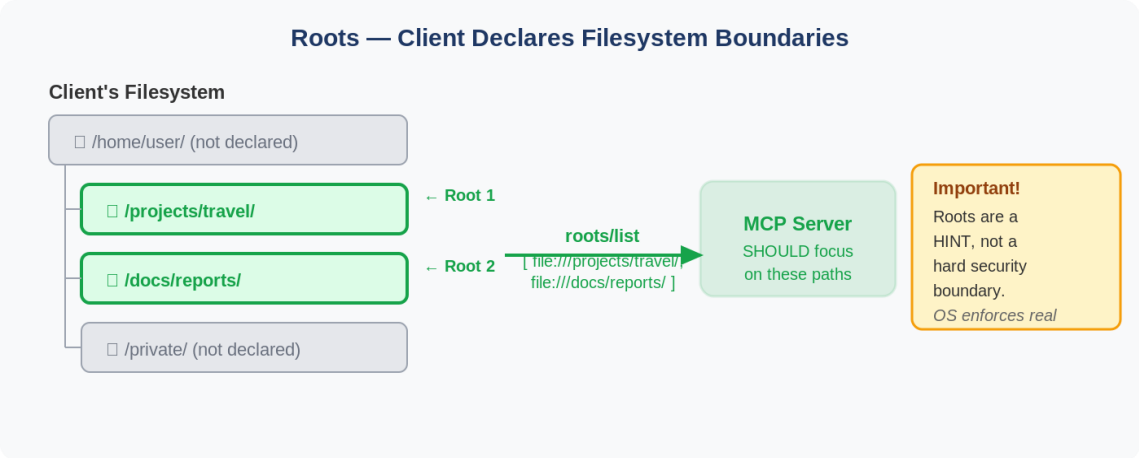
Clients must warn users about suspicious requests and let them review data before submitting.

Aspect	Elicitation	Sampling
Purpose	Ask the **user** for additional information	Ask an **LLM** to generate a response
Direction	MCP Server → Application/Agent → User	MCP Server → Application/Agent → LLM
Who provides the answer?	Human user	Language Model
Typical Use Case	Missing information, confirmation, approvals	AI-powered reasoning, summarization, generation
Output	User input	Model-generated text

3. Roots

Roots are how a client communicates which directories a server should focus on. They are **file://** URIs that the client declares to help the server understand the relevant workspace.

How it looks



Example Root

```
{
  "uri":  "file:///Users/agent/travel-planning",
  "name": "Travel Planning Workspace"
}
```

Key Facts about Roots

- **Always file:// URIs** — roots always point to local filesystem paths, never HTTP URLs.
- **Dynamic updates** — if the user opens a new folder, the client sends a roots/list_changed notification so the server knows.
- **Coordination, not security** — the spec says servers SHOULD respect root boundaries, not MUST enforce them.

server.py — a tool that asks the client "which folders should I focus on?"

```
from mcp.server.fastmcp import MCPServer, Context
from mcp.server.session import ServerSession

mcp = MCPServer("travel-server")

@mcp.tool()
async def scan_workspace(
    ctx: Context[ServerSession, None],
) -> str:
    """List the workspace folders the client has granted, and what's in them."""
```

```
    # Server -> Client: "roots/list" request goes out here
    result = await ctx.session.list_roots()

    if not result.roots:
        return "Client declared no roots — no workspace to scan."

    lines = []
    for root in result.roots:
        # root.uri is always a file:// URI, never HTTP
        path = root.uri.path            # e.g. /Users/agent/travel-planning
        lines.append(f"- {root.name}: {root.uri}")

        # SHOULD respect boundaries: only operate inside this path
        # (coordination, not security — the spec doesn't enforce it)
        # e.g. itinerary_files = os.listdir(path)

    return "Granted workspace roots:\n" + "\n".join(lines)
```

client.py — declaring the roots (the client is the source of truth)

```
from mcp import ClientSession
from mcp.shared.context import RequestContext
from mcp.types import ListRootsResult, Root, ErrorData
from pydantic import FileUrl

async def list_roots_callback(
    context: RequestContext[ClientSession, None],
) -> ListRootsResult | ErrorData:
    # The client decides which directories the server may focus on.
    # Always file:// URIs — matching the example from the deck:
    return ListRootsResult(
        roots=[
            Root(
                uri=FileUrl("file:///Users/agent/travel-planning"),
                name="Travel Planning Workspace",
            ),
            Root(
                uri=FileUrl("file:///Users/agent/expense-reports"),
                name="Expense Reports",
            ),
```

```
    ]
  )
# Wire it in when creating the session:
# async with ClientSession(read, write, list_roots_callback=list_roots_callback) as session:
#     await session.initialize()
#     result = await session.call_tool("scan_workspace", {})
```

And the **dynamic updates** point from your deck — when the user opens a new folder, the client notifies the server, which can then re-fetch:

```
# Client side: tell the server the root list changed
await session.send_roots_list_changed()


# Server side: subscribe to that notification and re-pull
# (in low-level server API, handle "notifications/roots/list_changed",
#  then call session.list_roots() again)
```